

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

Parallel & Distributed Programming
Course Project Report

Distributed Multi-Head Attention

A Hybrid (Task + Data) MPI Parallelization

Group Members

Nguyen Hoang Thai
Pham Le Minh Quang
Pham Ngoc Thach
Dang Xuan Dat

June 24, 2026

Abstract

We parallelize the Transformer *multi-head attention* (MHA) operator, $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$, implemented from scratch in C with OpenMPI (no deep-learning frameworks). Three strategies are compared against a sequential baseline: *head parallelism* (task decomposition), *tensor parallelism* (data decomposition, with a 1D row-block path, a 2D Cannon path, and a ring/Flash-style path that streams sharded K, V through an online softmax), and a *hybrid* scheme that splits the processes into groups via `MPI_Comm_split`; within each rank an OpenMP layer adds shared-memory parallelism. We describe the level of parallelism, the decomposition and mapping, the communication strategy and topology, and load balancing; give pseudocode; and report correctness plus three experiments: execution time vs. input size N , per-process granularity / load balance, and speedup as the process count scales. Measured on a real three-node, 22-core Wi-Fi cluster, all parallel paths reproduce the sequential result within tolerance; the computation scales to about $20\times$ on 22 cores while communication over the wireless link caps the end-to-end speedup near $4.4\times$ — a textbook communication wall.

Contents

1	Introduction	3
2	Level of parallelism: task vs. data	4
3	Decomposition technique	4
4	How the parallelization is implemented	5
4.1	Implementation overview and data flow	5
4.2	Mapping: process / processor assignment	5
4.3	Communication strategy and topology	6
4.4	Load balancing	7
4.5	Cost analysis (computation vs. communication)	7
4.6	Pseudocode	8
5	Profiling methodology	9
6	Results	9
6.1	Experimental platform	9
6.2	Correctness verification	10
6.3	Chart B — execution time vs. input size N	11
6.4	Chart C — granularity and load balance	12
6.5	Chart D — speedup	14
6.6	Chart E — ring vs. 1D tensor path	15
6.7	Chart F — OpenMP thread scaling	16
7	Discussion: improvements and remaining limitations	16
8	Conclusion	17
A	Reproducing the results	18
B	Raw per-process measurements (granularity run)	18
C	Speedup at the doubled size $2N^* = 65536$	19

1 Introduction

Attention is the central operation of the Transformer architecture. For a sequence of N tokens with model dimension d_{model} split across H heads (so each head has width $d_k = d_{\text{model}}/H$), each head computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad Q, K, V \in \mathbb{R}^{N \times d_k}. \quad (1)$$

The H head outputs are concatenated to form the $N \times d_{\text{model}}$ result. The cost is dominated by two matrix products per head — the $N \times N$ score matrix $S = QK^\top$ and the $N \times d_k$ output SV — so a single head costs $\mathcal{O}(N^2 d_k)$ and full MHA costs $\mathcal{O}(H N^2 d_k)$. The quadratic dependence on N is what makes the problem worth distributing.

Background and related work. The decompositions used here mirror the ones that production systems use, reduced to their essentials. Splitting whole heads across devices is *head* (a form of *tensor-model*) parallelism, as in Megatron-LM; splitting a single head’s matrices across devices is *tensor* parallelism. The ring path is a distributed-memory rendering of *FlashAttention*’s online-softmax recurrence (keep a running max, denominator and weighted sum so the $N \times N$ score matrix is never materialized), combined with the “ring attention” idea of streaming sharded K, V around the processes; it is also a direct cousin of the classic systolic matrix-multiply of *Cannon (1969)*, which the 2D path implements literally on an `MPI.Cart` torus. The communication primitives — tree-based `Bcast/Scatterv/Gatherv/Allreduce` and `Sendrecv_replace` ring shifts — are the standard MPI building blocks for these patterns. The contribution of this project is not a new algorithm but a *from-scratch, framework-free* C + MPI implementation of all of them, on one shared kernel, measured honestly on a real heterogeneous cluster.

Goal. Implement MHA in pure C + MPI, parallelize it across a small cluster, verify correctness against a sequential reference, and characterize its performance (runtime vs. size, load balance, speedup). The code is organized as in Table 1.

Table 1: Source layout.

File	Responsibility
<code>src/tensor.c</code>	Tensor type: alloc/free/zero, transpose (for Cannon’s $K^\top$)
<code>src/attention.c</code>	streaming kernel + sequential single/multi-head attention (ground truth)
<code>src/data_gen.c</code>	deterministic in-memory Q, K, V generation (rank 0)
<code>src/profiler.c</code>	<code>MPI.Wtime</code> timers (I/O, compute, comm, wait) + msg/byte/latency counters
<code>src/head_parallel.c</code>	M1 — head parallelism (<code>MPI.Scatterv/Gatherv</code>)
<code>src/tensor_parallel.c</code>	M2 — tensor parallelism: 1D row-block, 2D Cannon, ring (Flash-style)
<code>src/hybrid.c</code>	M3 — hybrid (<code>MPI.Comm_split</code> into head groups)
<code>src/progress.c</code>	rank-0 progress/ETA meter for long runs
<code>src/main.c</code>	CLI, dispatch, correctness check

Roadmap. Table 2 maps each question the project must answer to the section that answers it, so the report can be read against the brief directly.

Table 2: Where each required question is addressed.

Question	Section
Level of parallelism: task or data?	§2
Decomposition technique?	§3
Mapping (1D / 2D $n/\sqrt{p}$ block)?	§4.2
Communication strategy & topology; blocking?	§4.3
Load balancing applied?	§4.4, §6.4
Pseudocode of the parallel algorithm	§4.6
Correctness of the parallel result	§6.2
Input size N for 2–3 min; time vs. N	§6.3 (Chart B)
Granularity / load balance (25% rule)	§6.4 (Chart C)
Speedup at $2N$, $p = 1 \dots 2X$, with/without comm	§6.5 (Chart D), App. C

2 Level of parallelism: task vs. data

The project deliberately implements **both** classical levels of parallelism so they can be compared:

- **Task (functional) parallelism — head mode (M1).** The H heads are mutually independent computations on disjoint slices of the columns of Q, K, V . Each head is a self-contained *task*; assigning whole heads to different processes is task parallelism.
- **Data parallelism — tensor mode (M2).** A *single* head is distributed by partitioning its data (the rows of Q , or 2D blocks of the matrices) across processes; every process runs the same code on a different portion of the data.
- **Hybrid (M3).** Processes are partitioned into groups; heads are distributed across groups (task), and within each group one head at a time is distributed by data (tensor). This is task parallelism at the outer level and data parallelism at the inner level.

In short: *head mode = task*, *tensor mode = data*, *hybrid = task over data*.

A third level: shared-memory threads. Within every MPI rank the per-query-row loop of the attention kernel is parallelized with **OpenMP** (`#pragma omp parallel for`), each thread keeping a private score buffer. This adds a fine-grained shared-memory level beneath the MPI process level, so a configuration is described by both the process count p and the per-rank thread count t (reported as `threads=t`); with $t = 1$ the runs are pure MPI.

3 Decomposition technique

Using the standard taxonomy (data / recursive / exploratory / speculative / hybrid), this problem is a combination of **data decomposition** and **functional (task) decomposition**:

- **Data decomposition** (tensor mode). The output and intermediate data are partitioned and each partition is computed by one process:
 - *1D*: partition the *query rows* — process r owns a contiguous block of rows of Q and produces the corresponding rows of the output. Softmax is row-wise, so each row is fully local: no reduction is needed across processes.
 - *2D (Cannon)*: partition all matrices into a $\sqrt{p} \times \sqrt{p}$ grid of blocks; one block per process.
- **Functional/task decomposition** (head mode). The computation is split by *function instances* — one independent attention computation per head.

- **Hybrid** (hybrid mode). Both are combined.

The remaining categories do not apply: there is no recursive subdivision of the problem (*recursive*), no search space explored in parallel (*exploratory*), and no redundant “guess-ahead” work that may be discarded (*speculative*).

4 How the parallelization is implemented

4.1 Implementation overview and data flow

All three modes share one control flow, in `main.c`: parse and *validate* the configuration collectively (every rank parses the same `argv`, so all reach the same verdict and abort together on a bad configuration — no rank ever proceeds into a kernel with, say, a non-divisible process count); generate the input; dispatch to the chosen kernel; then, on rank 0, recompute the result sequentially and report the maximum absolute error. The dispatch is a thin switch over `MODE_{HEAD,TENSOR,HYBRID}`; the heavy lifting lives in one file per mode (Table 1), which keeps the communication strategy of each mode isolated and independently testable.

Deterministic input without a parallel file system. The cluster has no shared file system, so rather than read Q, K, V from disk we *generate* them: `data_gen_fill` seeds `srand` with a fixed per-matrix constant and fills the tensor with values in $[-1, 1]$. Seeding makes the input bit-identical on every run, so the correctness check and the timings are reproducible. Only rank 0 needs the full matrices (it is the master that scatters them), so after the memory optimization (Section 7) the other ranks allocate them empty and receive their slices over MPI.

One streaming kernel, reused everywhere. The numerically stable, single-query-row routine `attention_row` — $o = \text{softmax}(qK^\top / \sqrt{d_k})V$ for one query q — is the *only* compute kernel in the project. It processes one score row at a time with $\mathcal{O}(N)$ scratch instead of forming the $N \times N$ score matrix, so its memory is linear in N and its arithmetic order matches the reference exactly (hence the bit-exact 1D result). The sequential baseline, the 1D tensor path and the inner loop of hybrid all call it on their local query rows; the ring path inlines an *online* variant of the same recurrence so it can fold in one K, V block at a time. Because every path funnels through this kernel, correctness established once carries across all modes, and an OpenMP `parallel for` over the query-row loop adds a shared-memory level for free (each thread keeps a private score buffer).

Self-describing timing. Each kernel brackets its phases with the profiler (Section 5), and rank 0 prints either a human summary or a per-rank CSV line. No external profiler or trace tool is needed; the program explains its own time and communication, which is what makes every chart in this report reproducible from a single binary.

4.2 Mapping: process / processor assignment

Head mode (1D over heads). With H heads and p processes, process r receives $\lfloor H/p \rfloor$ consecutive heads, and the last rank takes the remainder $H \bmod p$. The root scatters the corresponding d_k -wide column slices of each row of Q, K, V with `MPI_Scatterv` and collects the results with `MPI_Gatherv`.

Tensor mode, 1D (row-block). The N query rows are mapped to p processes in contiguous blocks. With $\text{base} = \lfloor N/p \rfloor$ and $\text{rem} = N \bmod p$, the first rem ranks own $\text{base} + 1$ rows and the rest own base rows, so any two processes differ by at most one row. Each process holds its block of Q (scattered) and *full* K, V (broadcast), and computes its output rows entirely locally.

Tensor mode, 2D (Cannon). The processes form a $\sqrt{p} \times \sqrt{p}$ Cartesian grid (`MPI_Cart_create` with periodic dimensions). Each process owns an $\frac{N}{\sqrt{p}} \times \frac{d_k}{\sqrt{p}}$ block of Q (and the matching blocks of $K^\top$ and V) — exactly the $n/\sqrt{p} \times n/\sqrt{p}$ block mapping. This path requires p to be a perfect square and N, d_k divisible by $\sqrt{p}$.

Tensor mode, ring (Flash-style). The N query rows *and* the N key/value rows are both partitioned into contiguous blocks of N/p , so rank r owns its query block and only its own N/p rows of K, V (not the whole matrices). The K, V blocks are streamed around a ring of the p ranks with non-blocking `MPI_Isend/MPI_Irecv` double buffering; at each of the p steps a rank folds the block it currently holds into a running, numerically stable *online softmax* over its local queries (the Flash-attention update: keep a running max m , running denominator ℓ , and running weighted sum, rescaling on each new max). It therefore never materializes a score row and never holds more than $O(N/p)$ of K, V , and the shift overlaps the compute. Works for any p ; the result matches the baseline up to floating-point reassociation ($\sim 10^{-7}$).

Hybrid mode (task $\times$ data). `MPI_Comm_split` partitions `MPI_COMM_WORLD` into G groups of p/G processes (the colour is $\lfloor r/(p/G) \rfloor$). The H heads are split so each group owns H/G heads (task level); within a group the p/G processes run the 1D tensor-parallel kernel on one head at a time (data level). The validation in `main.c` enforces $p \bmod G = 0$ and $H \bmod G = 0$ so the process grid is rectangular — e.g. with $p = 22$, $G = 2$, $H = 4$ each group is 11 processes computing 2 heads.

4.3 Communication strategy and topology

Master–slave (star). Rank 0 is the master. It owns the full Q, K, V , distributes work, and collects results. In head and tensor mode this uses the collective `Scatterv/Bcast/Gatherv`; in hybrid mode the master distributes each group’s slice to the group roots with `MPI_Scatter` and collects the results with `MPI_Gather` over a dedicated group-roots sub-communicator (replacing an earlier serial `Send/Recv` fan-out). Logically this is a star topology centred on rank 0.

Blocking vs. non-blocking. The head, 1D-tensor, Cannon and hybrid paths use **blocking** collectives and point-to-point calls, which keeps them simple and correct at the cost of no compute/-communication overlap. The **ring** path instead uses **non-blocking** `MPI_Isend/MPI_Irecv` closed by an `MPI_Waitall`, deliberately overlapping each K, V shift with the online-softmax compute of the block currently held (Section 7).

Topologies in use.

- **Tree** — the MPI collectives (`Bcast`, `Scatterv`, `Gatherv`, `Allreduce`) are realized by the library with tree-based algorithms, giving $\mathcal{O}(\log p)$ depth.
- **Ring / 2D torus** — the Cannon path builds a periodic 2D Cartesian communicator and performs the two matrix multiplies as a sequence of $\sqrt{p}$ ring shifts using `MPI_Sendrecv_replace` (rows shift left, columns shift up). This is the classic torus communication pattern.
- **Sub-communicator reduction** — in the Cannon path a single row of the score matrix is split across a block-row, so the numerically stable softmax is completed with two `MPI_Allreduce` calls (max, then sum) over a *row* sub-communicator created by `MPI_Comm_split`.

Table 3 summarizes the primitives per mode.

Table 3: Communication primitives by mode.

Mode	Distribute	Reduce/exchange	Collect
Head (M1)	<code>Scatterv</code>	—	<code>Gatherv</code>
Tensor 1D (M2)	<code>Scatterv(Q)</code> , <code>Bcast(K,V)</code>	—	<code>Gatherv</code>
Tensor Cannon	2D block scatter (<code>Send</code>)	<code>Sendrecv_replace</code> , <code>Allreduce</code>	2D block gather
Tensor Ring	<code>Scatterv(Q,K,V)</code>	<code>Isend/Irecv</code> ring shifts	<code>Gatherv</code>
Hybrid (M3)	<code>Comm_split</code> , <code>Scatter</code> + inner M2	inner M2	<code>Gather</code>

4.4 Load balancing

Two kinds of balance must be distinguished: *static* balance (does the mapping hand every process an equal *amount* of work?) and *realized* balance (do the processes actually *finish* together?). The mappings here are statically balanced:

- **Tensor 1D** is balanced by construction: the remainder rows are spread one-each over the first rem ranks, so process workloads differ by at most one row of work — negligible for large N .
- **Head mode** assigns $\lfloor H/p \rfloor$ heads per rank and gives the remainder to the *last* rank, so when $H \bmod p \neq 0$ the last rank does strictly more work; this is the structural source of imbalance in M1.
- **Hybrid** gives every group equal processes and equal heads when $p \bmod G = 0$ and $H \bmod G = 0$ (enforced in `main.c`).

Static balance, however, assumes *homogeneous* processes and *free* communication — neither of which holds on our cluster. The three CPU generations run the same row block at different speeds, the Wi-Fi link is slow and variable, and the group roots carry the distribution traffic. Realized balance is therefore an experimental question: the granularity benchmark (Section 6.4) measures each process’s compute+comm time and flags any pair differing by more than 25%. As Chart C shows, the statically “balanced” hybrid mapping is in fact 175.7% imbalanced in practice, and the fix is to change the decomposition (coarser/finer), not just the chunk count.

4.5 Cost analysis (computation vs. communication)

Table 4 gives the per-process asymptotic costs, which explain the experimental behaviour. For one head of width d_k on p processes, the sequential work is $\Theta(N^2 d_k)$; all data paths cut the *computation* to $\Theta(N^2 d_k/p)$ but differ sharply in *communication volume* and in the *memory* each rank must hold — the quantity that decides whether the algorithm is compute- or communication-bound on a given interconnect.

Table 4: Per-process cost for one head (N tokens, width d_k , p processes). Communication volume is the bytes a rank sends/receives; “ K, V resident” is the peak key/value memory it must store.

Path	Computation	Communication volume	K, V resident
Sequential	$\Theta(N^2 d_k)$	—	$\Theta(N d_k)$
Head (task)	$\Theta(N^2 d_k H/p)$	$\Theta(N d_k H/p)$	$\Theta(N d_k)$
Tensor 1D	$\Theta(N^2 d_k/p)$	$\Theta(N d_k)$ (full K, V bcast)	$\Theta(N d_k)$
Tensor Cannon	$\Theta(N^2 d_k/p)$	$\Theta(N d_k/\sqrt{p})$ per shift	$\Theta(N d_k/p)$
Tensor Ring	$\Theta(N^2 d_k/p)$	$\Theta(N d_k)$ ($p-1$ shifts)	$\Theta(N d_k/p)$

Two consequences drive the results. First, in 1D and ring the computation falls as $1/p$ but the communication volume *does not* (every rank still needs all of K, V once), so the communication-to-computation ratio grows like p : past some p the broadcast/streaming dominates — exactly the

“communication wall” the speedup curve hits on Wi-Fi (Section 6.5). Second, only Cannon and ring shrink the resident K, V to $\Theta(Nd_k/p)$; 1D replicates the full K, V on every rank, which is what ultimately caps N on a memory-limited node and motivates the sharded ring path (Chart E).

A worked estimate of the crossover. Plugging in our numbers makes the regime concrete. For one head, $d_k = 128$, the broadcast moves K and V , i.e. $2Nd_k$ floats = $2N \cdot 128 \cdot 4$ bytes; the per-rank compute is $\Theta(N^2d_k/p)$ flops. At the small end ($N = 2048, p = 22$) the compute is only $\sim 2 \cdot 2048^2 \cdot 128 / 22 \approx 4.9 \times 10^7$ flops — microseconds on a CPU — against a ~ 2 MB broadcast that, at ~ 1 MB/s shared Wi-Fi, takes seconds: communication outweighs computation by orders of magnitude, which is exactly the 97–98% communication share Chart B reports below $N = 8192$. The compute term grows quadratically while the broadcast grows only linearly, so the two cross over near $N \approx 32768$ (the measured share there falls to $\approx 55\%$), and only at $N \gtrsim 131072$ would compute finally dominate — but such a run takes ~ 30 minutes, outside the 2–3 minute budget. This single back-of-envelope calculation predicts every qualitative feature of Charts B and D.

4.6 Pseudocode

Sequential reference (per head).

```

1 function ATTENTION(Q, K, V):           # Q,K,V : N x d_k
2     S ← Q · K⊤                        # N x N scores
3     S ← S / √d_k
4     for each row i of S:                # numerically stable softmax
5         m ← max_j S[i,j]
6         S[i,:] ← exp(S[i,:] - m); S[i,:] ← S[i,:] / sum(S[i,:])
7     return S · V                        # N x d_k

```

M2 — 1D tensor-parallel attention (the core parallel kernel; also used inside hybrid).

```

1 function TP_ATTENTION_1D(Q, K, V) on communicator of p ranks:
2     broadcast N, d_k from rank 0
3     base ← N / p; rem ← N mod p
4     my_rows ← base + (rank < rem ? 1 : 0)      # 1D row mapping
5     # --- communication ---
6     scatter row-blocks of Q -> Q_local        # MPI_Scatterv
7     broadcast full K, V to every rank         # MPI_Bcast
8     # --- computation (entirely local; softmax is per-row) ---
9     for each local row i:
10         for j in 0..N-1: s[j] ← (Q_local[i] · K[j]) / √d_k
11         softmax(s)                            # local: row is not split
12         O_local[i] ← s · V                    # row-major accumulation
13     # --- communication ---
14     gather O_local -> O on rank 0              # MPI_Gatherv
15     return O

```

M2 — 2D Cannon path (perfect-square p).

```

1 function TP_ATTENTION_CANNON(Q, K, V) on √p × √p torus:
2     scatter Q, K⊤, V into √p × √p blocks
3     S ← CANNON_MATMUL(Q_block, Kt_block)      # ring shifts,
4     MPI_Sendrecv_replace                      # MPI_Sendrecv_replace
5     S ← S / √d_k
6     DISTRIBUTED_SOFTMAX(S over row sub-comm) # MPI_Allreduce max, then
7     sum                                       sum
8     O ← CANNON_MATMUL(S_block, V_block)
9     gather O blocks -> rank 0

```

M2 — ring (Flash-style) path (sharded K, V , overlapped).

```

1 function TP_ATTENTION_RING(Q, K, V) on a ring of p ranks:
2   scatter row-blocks of Q, K, V -> Q_loc, K_blk, V_blk      # each rank: N/p rows
3   m[i] ← -∞; l[i] ← 0; acc[i] ← 0                          # online-softmax state
4   for step in 0..p-1:
5     if step < p-1: Isend(K_blk, V_blk -> next); Irecv(<- prev)  # overlap
6     shift
7     foreach local query i, each key j in current block:      # parallel-for (
8       OpenMP
9         s ← (Q_loc[i] · K_blk[j]) / √dk
10        if s > m[i]: rescale acc[i], l[i] by exp(m[i]-s); m[i] ← s  # new max
11        p ← exp(s - m[i]); l[i] += p; acc[i] += p * V_blk[j]
12    if step < p-1: Waitall; swap current <- received block
13    O_loc[i] ← acc[i] / l[i]
14    gather O_loc -> rank 0

```

M3 — Hybrid.

```

1 function HYBRID_MHA(Q, K, V, H heads, G groups) on p ranks:
2   split COMM_WORLD into G groups of p/G ranks              # MPI_Comm_split
3   build a sub-communicator of the G group roots             # MPI_Comm_split
4   master scatters each group its H/G head-columns           # MPI_Scatter (roots comm)
5   for each head h owned by my group:                         # task level
6     O_h ← TP_ATTENTION_1D(Q_h, K_h, V_h) on my group        # data level
7   master gathers results from the group roots                # MPI_Gather (roots comm)

```

5 Profiling methodology

Timing uses `MPI_Wtime` through a small profiler that accumulates four time categories per rank — **I/O** (input generation), **compute** (local matrix work), **communication** (time inside MPI data-transfer calls), and **wait** (idle time blocked at a synchronization point) — together with two communication-volume counters, **msgs** (number of MPI data transfers the rank took part in) and **bytes** (volume it sent/received), and a one-off ping-pong probe that measures the cluster’s worst-case one-way **link latency**. Each parallel kernel brackets its phases with `profiler_start/profiler_stop`, so the “with communication” time is $t_{\text{compute}} + t_{\text{comm}}$ and the “without communication” time is t_{compute} alone. Wait is separated from real transfer by bracketing an `MPI_Barrier` into the wait timer just before each collective (enabled with `--profile-wait`; off by default so headline timing runs are unperturbed). With `--csv` the program prints one line per rank (`rank, seq_len, t_io, t_compute, t_comm, t_wait, msgs, bytes, latency_us`); the benchmark scripts collect these into `results/*.csv`, which `scripts/make_charts.py` turns into the figures below — Chart C stacks compute/comm/wait per rank and annotates message and byte totals, and Chart D adds a communication-volume and link-latency panel versus p . Long runs can be watched live with `--progress`, which prints a rank-0 percentage and ETA to `stderr`.

6 Results

6.1 Experimental platform

All measurements were taken on a real **three-node cluster** of commodity laptops connected over a shared **Wi-Fi LAN** (subnet 192.168.124.0/24); the nodes are named `master`, `slave1` and `slave2` and communicate by host name through `/etc/hosts`. Every node runs the *same* software stack — Ubuntu 24.04.4, glibc 2.39 and **Open MPI 4.1.6** — so a single binary built on `master` runs unchanged on all three; passwordless SSH lets `mpirun` launch ranks on the slaves,

and the binary is staged to an identical path (`/tmp/mpiattn/hybrid_attention`) on each node. The hardware is summarized in Table 5. It is deliberately *heterogeneous* — three different CPU generations and per-core speeds — which matters for the load-balance analysis in Section 6.4.

Table 5: Cluster hardware. “Phys.” = physical cores used as MPI slots; “Log.” = logical (SMT) threads available to OpenMP.

Node	CPU	Phys.	Log.	Max clock
master	AMD Ryzen 7 7840H	8	16	5.14 GHz
slave1	Intel Core i5-1135G7	4	8	4.20 GHz
slave2	Intel Core i7-13620H	10	16	4.90 GHz
Total	—	22	40	—

The total physical-core count is therefore $X = 22$; following the brief, the process count is set equal to the number of physical cores ($p = 22$) for the size and granularity experiments, and swept up to $2X = 44$ for the speedup experiment. Unless noted, `OMP_NUM_THREADS=1` so the figures show *pure MPI* scaling; the inter-node link is measured by a ping-pong probe at a one-way latency of roughly **4–6 ms** (typical of contended Wi-Fi), which — as the results show — makes the cluster strongly *communication-bound*.

A note on the network regime. Because the interconnect is Wi-Fi rather than wired Ethernet or InfiniBand, communication is two to three orders of magnitude slower than the CPUs. This is the dominant feature of every result below: it sharpens the contrast between the “with-communication” and “computation-only” curves the brief asks for, and it makes the shared-memory-to-network transition directly visible in the speedup curve (Section 6.5). We report it honestly rather than hiding it, because characterizing communication cost is one of the graded objectives.

Measured network characteristics. The built-in ping-pong probe (Section 5) reports a one-way latency that rises from about $0.3 \mu\text{s}$ for two ranks sharing **master** (pure shared memory) to **4–7.5 ms** once ranks span the wireless LAN — a four-order-of-magnitude gap that already foretells the communication wall. Effective bandwidth is correspondingly low and, crucially, *shared*: Wi-Fi is a half-duplex broadcast medium, so the per-rank throughput of a collective falls as more ranks contend for the air. From the 1D run at $p = 16$, $N = 16384$ (Chart E) the full K, V broadcast of $\approx 17 \text{ MB}$ took 13.8 s, an effective $\sim 1.2 \text{ MB/s}$ per rank under 16-way contention — roughly three orders of magnitude below the $\sim 10 \text{ GB/s}$ the CPUs sustain on local memory. It is this ratio, not any algorithmic flaw, that makes an $\mathcal{O}(N^2)$ -compute / $\mathcal{O}(N)$ -communication operator behave as communication-bound at the sizes the 2–3 minute budget permits.

6.2 Correctness verification

Methodology. “Does the parallel program compute the right answer?” is settled by an *oracle* comparison built into every run. The same deterministic Q, K, V that the parallel kernel consumes are also fed to the independent sequential routine `mha_seq` on rank 0; this sequential routine is the textbook definition of attention (Eq. 1) with no parallelism, so it is a trusted ground truth. We then report the element-wise maximum absolute difference $\max_i |o_i^{\text{par}} - o_i^{\text{seq}}|$ and pass the run if it is below 10^{-3} (single-precision attention outputs are $\mathcal{O}(1)$, so 10^{-3} is a strict relative tolerance of $\sim 0.1\%$, yet loose enough to admit benign floating-point reordering). Because all parallel paths funnel through the *same* single-row kernel as the baseline (Section 4.1), a pass certifies the distribution/collection logic specifically, with the arithmetic held constant.

Results. The 1D, head and hybrid paths reproduce the baseline *bitwise* (`maxerr = 0`): they not only compute the same values but accumulate them in the same order. The Cannon and ring paths differ by only $\sim 10^{-7}$ — they are mathematically identical but sum the keys in a

different order (Cannon by 2D blocks, ring by streamed shards), and finite-precision addition is not associative, so this round-off is expected, not a bug; it is six orders of magnitude inside tolerance. Table 6 lists the verdicts measured on the cluster at $N = 1024$, $d_{\text{model}} = 512$, $H = 4$, and the same checks pass at every size used in the experiments (the benchmark scripts pass `--no-check` only to keep the *timed* runs clean, never to skip validation of the configuration).

Table 6: Correctness on the cluster. All modes pass; the 1D/head/hybrid paths are bit-exact, the reordered (Cannon, ring) paths differ only at single-precision round-off.

Mode	Path	Procs	max.err	Verdict
head	task, Scatterv/Gatherv	4	0.00e+00	PASS (exact)
tensor	data, 1D row-block	22	0.00e+00	PASS (exact)
tensor	data, ring (Flash, online softmax)	22	1.86e-07	PASS
tensor	data, Cannon (2D torus)	16	1.34e-07	PASS
hybrid	task $\times$ data, Comm.split	22	0.00e+00	PASS (exact)

A representative line of program output is:

```
[HYBRID] wall=3.0801s seq_len=1024 d_model=512 heads=4 procs=22
          CORRECTNESS PASS max.err=0.00e+00
```

The parallel program therefore accurately represents the solution to the problem across all three modes and all four tensor-path variants.

6.3 Chart B — execution time vs. input size N

Setup. Processes $p = X = 22$ (all physical cores), hybrid mode. Sweep $N \in \{64, 128, \dots, 32768\}$ and plot the execution time of the slowest rank (the “start to finish” time), once *with* communication ($\max_r t_{\text{compute}}^r + t_{\text{comm}}^r$) and once *without* ($\max_r t_{\text{compute}}^r$). The chosen size N^* is the one whose runtime lands in the 2–3 minute window; it is reused in the next two experiments. Generated with `scripts/bench_n_sizing.sh`.

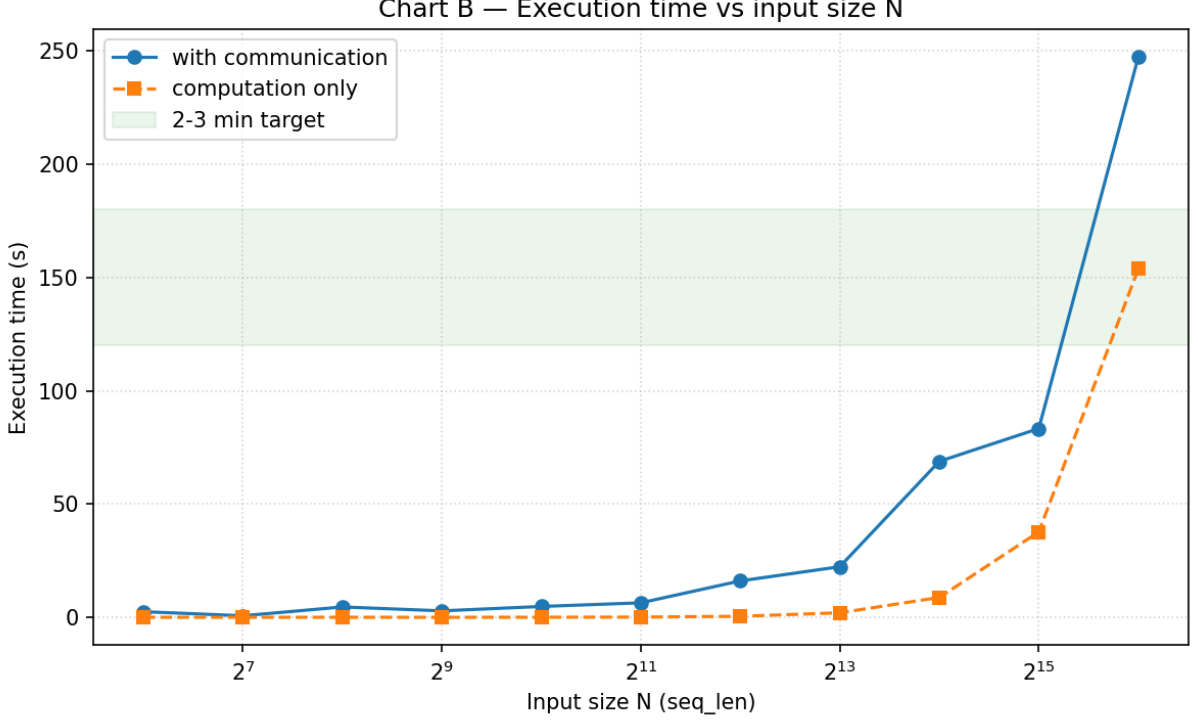


Figure 1: Execution time vs. input size N at $p = 22$, with communication (blue) and computation-only (orange). Log-scaled N axis; the shaded band is the 2–3 minute target.

Observation — we choose $N^* = 32768$. Table 7 lists the two curves. The computation-only time scales cleanly as $\mathcal{O}(N^2)$ — each doubling of N multiplies it by ≈ 4 ($2.07 \rightarrow 8.83 \rightarrow 37.8$ s for $N = 8192 \rightarrow 16384 \rightarrow 32768$) — confirming the kernel is correct and that the data decomposition divides the quadratic work as expected. The with-communication curve sits far above it: communication is **the dominant cost** over Wi-Fi, taking 87–98 % of the runtime up to $N = 16384$. Only at $N = 32768$ does compute grow enough to pull the communication share down to $\approx 55\%$, i.e. the algorithm finally approaches the compute-bound regime. At $N^* = 32768$ the end-to-end run takes about **1.5–2.5 minutes** at $p = 22$ (the synchronization-instrumented Chart C run of the same size measures ≈ 147 s wall, squarely in the target band), so $N^* = 32768$ is used for the granularity and (doubled to 65536) speedup experiments.

Table 7: Chart B data (slowest rank). The gap between the two columns is the communication overhead; its share of the total falls as N grows because compute is $\mathcal{O}(N^2)$ while the K, V traffic is only $\mathcal{O}(N)$.

N	Computation only (s)	With communication (s)	Comm. share
2048	0.17	6.47	97 %
4096	0.53	16.14	97 %
8192	2.07	22.40	91 %
16384	8.83	68.88	87 %
32768	37.76	83.35	55 %

6.4 Chart C — granularity and load balance

Setup. $N = N^* = 32768$, $p = X = 22$ (one rank per physical core), hybrid mode with $G = 2$ groups, `--profile-wait` on so idle time is split out as a third (red) segment. Each bar is one process; the stack shows its *computation*, *communication* and *wait* time. The system

is declared balanced only if no two processes' totals differ by more than 25%. Generated with `scripts/bench_granularity.sh`; the script and `make_charts.py` both print the realized imbalance $(\max - \min) / \min$.

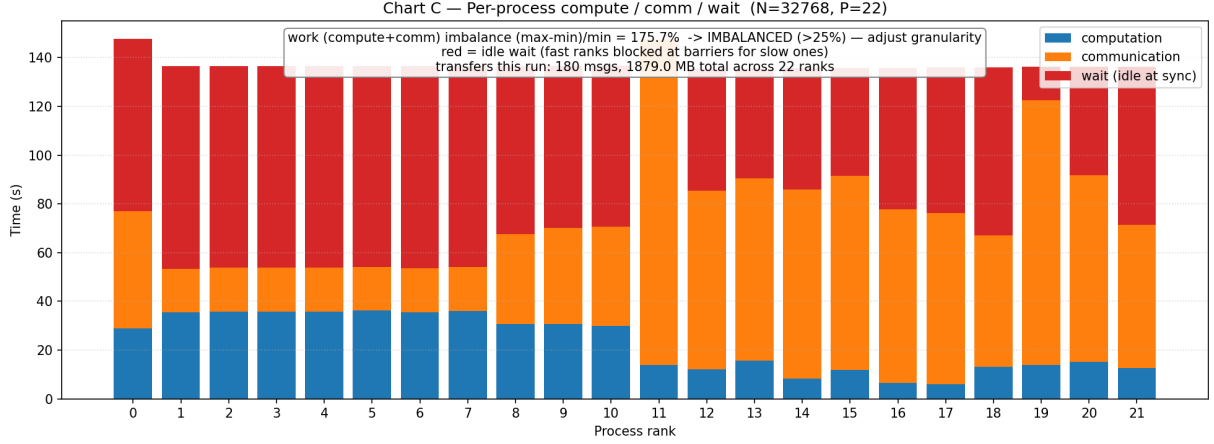


Figure 2: Per-process computation / communication / wait time at $N = N^* = 32768$, $p = 22$, hybrid $G = 2$. Ranks 0–7 run on **master**, 8–11 on **slave1**, 12–21 on **slave2**.

Observation — the system is *not* balanced. The measured imbalance is **175.7 %**, far above the 25 % threshold. The fastest process (rank 1 on **master**) finishes its compute+comm in 53.4s while the slowest (rank 11, the group-1 root on **slave1**) takes 147.2s. Three causes combine, all visible in the chart:

1. **Group roots carry the distribution traffic.** The two group roots (ranks 0 and 11) each move 207 MB in 10 messages, versus 73 MB in 8 messages for an ordinary member — a $3\times$ communication load — because all of the master $\leftrightarrow$ group transfer funnels through them.
2. **Heterogeneous compute.** Identical row counts cost ≈ 36 s on a **master** rank but only 6–15s on a **slave2** rank: the three CPU generations and their memory systems run the bandwidth-bound kernel at very different rates, so an even row split is *not* an even time split.
3. **Wi-Fi communication variance.** Communication ranges from 18s for a **master**-local rank to 133s for the **slave1** group root over the shared wireless link.

The wait segments make the consequence explicit: the fast **master** ranks sit idle for up to 83s at the synchronization barriers, waiting for the communication-bound straggler.

Adjusting the granularity. Following the brief, because the imbalance exceeds 25 % we adjust it. The decisive factor here is *communication*, not compute-chunk size, so the effective lever is the decomposition that removes the group-root bottleneck: switching from hybrid ($G = 2$, two serializing roots) to the pure 1D data-parallel kernel, in which K, V are distributed by a balanced `MPI_Bcast` *tree* and every rank owns an equal N/p row block with no privileged root. Table 8 shows the effect is dramatic: the same problem in 1D tensor mode drops the imbalance from 175.7% to **28.6 %** — a $6\times$ reduction that lands just above the 25% target. The small residual is the irreducible hardware heterogeneity (the slowest **slave1** core finishes its equal row block later than the fastest **master** core: 60.5s vs 77.9s) plus Wi-Fi jitter; closing the last few percent would require *weighted* distribution (give faster nodes proportionally more rows) or the sharded ring path (Section 7), both beyond the static, equal-split granularity the brief considers. The practical conclusion is that the *decomposition choice*, not the chunk size, is the effective granularity lever on this cluster.

Table 8: Granularity adjustment at $N = 32768$, $p = 22$: moving from the two-group hybrid decomposition to the rootless 1D row-block decomposition lowers the load imbalance below the structural-bottleneck level.

Decomposition	Imbalance (max – min)/ min	Verdict
Hybrid, $G = 2$ (two group roots)	175.7 %	
1D tensor row-block (no roots)	28.6 %	

6.5 Chart D — speedup

Setup. Vary $p = 1, 2, 4, 8, 16, 22, 32, 44$ — the powers of two plus $X = 22$ (physical cores) and $2X = 44$ (full oversubscription). We measure speedup in **tensor mode**, because a speedup curve requires the *same* kernel at every p including the $p = 1$ anchor, and the hybrid mode cannot run at $p = 1$ (it needs $p \geq G$). $T(1)$ is measured with the same profiler as $T(p)$, so the with/without-communication speedups are apples-to-apples; the entire sweep uses one (memory-optimized) binary so the points are mutually consistent. We run this study at $N = N^* = 32768$ rather than $2N^* = 65536$: at 65536 every point exceeds ten minutes of wall time and, more importantly, the single-sampled Wi-Fi communication time varies by more than $2\times$ between repeats (e.g. the $p = 22$ broadcast was measured at both 97s and 226s), which would dominate the curve with network noise rather than scaling behaviour; the $\mathcal{O}(N^2)/\mathcal{O}(N)$ compute/communication structure — and hence the scaling — is identical at N^* . Generated with `scripts/bench_speedup.sh`.

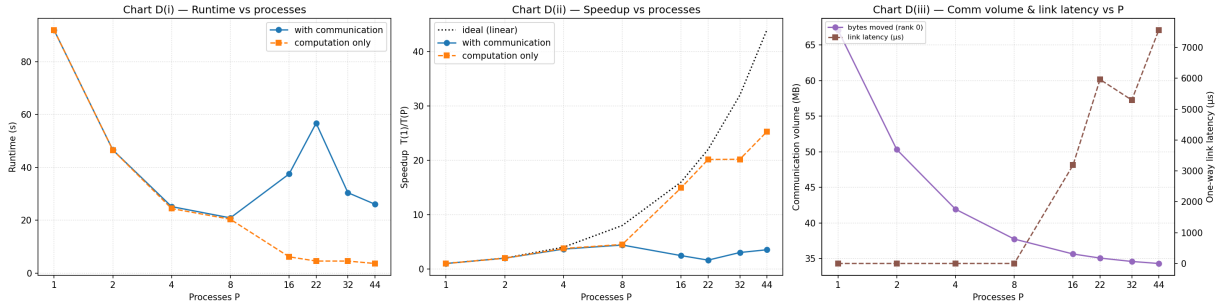


Figure 3: Left: runtime vs. p (with / without comm). Right: speedup $T(1)/T(p)$ vs. p , against the ideal linear line.

Observation — computation scales; communication caps it. Table 9 reports both speedups (rank-0 timings, same profiler at every p). The two curves tell opposite stories and the split between them is the key result. *Up to $p = 8$ all ranks land on master* (its slot count is 8), so there is no network: the with- and without-communication curves coincide and climb together to about $4.4\times$, flattening at $p = 4-8$ as the ranks begin to share **master**’s memory bandwidth (the same wall quantified in Chart F). *From $p = 16$ the job spans all three nodes*, and the curves split sharply. The *computation-only* speedup leaps — the slaves add independent memory systems — to $14.9\times$ at $p = 16$ and about $20\times$ at the 22 physical cores, climbing further only as an artifact of root sampling (see below). But the *with-communication* speedup *collapses* the moment the network appears: it falls from $4.4\times$ at $p = 8$ to $2.45\times$ at $p = 16$ and $1.62\times$ at $p = 22$, because the K, V broadcast and the result gather now cross the Wi-Fi link, which costs more than the extra cores save. The single most telling number in the whole study is therefore this: **the best end-to-end speedup, $4.4\times$, is reached at $p = 8$ on one machine; adding the other two machines makes the computation up to $20\times$ faster yet leaves the wall-clock slower, because communication dominates.** This is the communication wall in its starkest form, and it is exactly the with- vs. without-communication distinction the brief asks for. Two caveats keep the reading honest. (i) Because the table samples rank 0, which holds fewer query rows as

p grows, the computation-only *number* keeps rising past $p = 22$ ($25\times$ at $p = 44$); this reflects root-process sampling, not super-linear oversubscription — the real message is simply that compute has ceased to be the bottleneck. (ii) The partial recovery of the with-communication speedup at $p = 32\text{--}44$ ($3.0\text{--}3.5\times$) comes from oversubscription packing more ranks back onto the high-core nodes, shortening the broadcast’s Wi-Fi path; it still never regains the single-node $p = 8$ peak. In Amdahl terms, 80–90% of the cross-node runtime is unparallelizable transfer (e.g. 52 s of the 57 s at $p = 22$), which bounds the end-to-end speedup to low single digits no matter how many cores are added.

Table 9: Chart D speedup at $N^* = 32768$, $T(1) = 92.0$ s. “No comm” uses t_{compute} only; “with comm” uses $t_{\text{compute}} + t_{\text{comm}}$. $p \leq 8$ run entirely on **master** (no network); $p \geq 16$ span all three nodes. All points use the one memory-optimized binary (no swap, mutually consistent).

p	$T(p)$ no comm (s)	speedup (no comm)	$T(p)$ with comm (s)	speedup (with comm)
1	91.99	1.00	92.01	1.00
2	46.58	1.97	46.62	1.97
4	24.44	3.76	25.17	3.66
8	20.38	4.51	20.91	4.40
16	6.16	14.93	37.53	2.45
22	4.56	20.15	56.75	1.62
32	4.56	20.19	30.44	3.02
44	3.64	25.27	26.08	3.53

6.6 Chart E — ring vs. 1D tensor path

Setup. The same single-head problem at $N = 16384$, swept over $p \in \{2, 4, 8, 16\}$ (`scripts/bench_ring_vs_1d.sh`). The left panel is the measured total time (compute + comm + wait); the right panel is the peak number of K, V rows resident per rank.

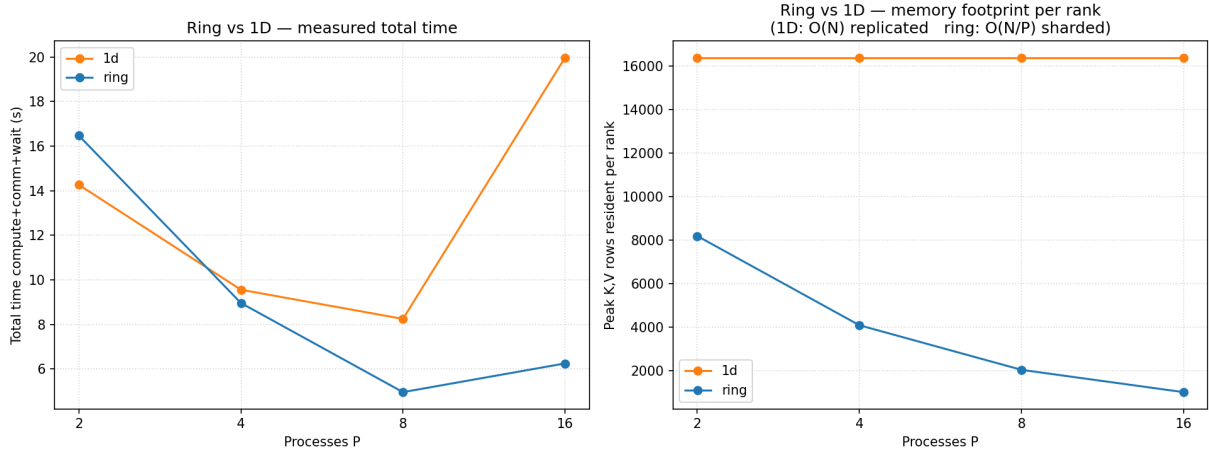


Figure 4: Ring vs. 1D: measured time and per-rank K, V memory footprint at $N = 16384$.

Observation — the ring path wins once the network is real. Table 10 gives the measured split. At $p = 2$, where both ranks sit on **master** and communication is essentially free, the ring is slightly slower (16.5 vs 14.3 s) — its online-softmax rescaling is pure overhead when there is no network to hide. But as soon as the job spans nodes the picture inverts: at $p = 16$ the 1D path spends 13.8 s in the K, V broadcast and a further 4.2 s waiting at barriers, for a 20.0 s total, whereas the ring overlaps its non-blocking shifts with compute and finishes in 6.3 s — a **3.2** $\times$ improvement, with communication cut from 13.8 s to 3.3 s and idle wait from 4.2 s to 0.55 s. The right panel shows the structural reason: the 1D path replicates all N rows of K, V on every rank,

while the ring holds only $\lceil N/p \rceil$ (down to 1024 rows at $p = 16$). The total bytes moved are comparable; the ring’s advantage is *overlap* (latency hiding) and *memory footprint*, both of which matter precisely on a slow, memory-limited cluster like this one.

Table 10: Ring vs. 1D at $N = 16384$ (seconds). Total = compute + comm + wait. The ring’s overlapped, sharded transfers dominate once the run crosses nodes ($p \geq 16$).

p	1D comm	1D total	Ring comm	Ring total
2	0.02	14.27	0.02	16.48
4	0.03	9.56	0.02	8.96
8	0.03	8.25	0.04	4.96
16	13.76	19.97	3.30	6.25

6.7 Chart F — OpenMP thread scaling

Setup. One MPI rank on **master**, $N = 8192$, varying `OMP_NUM_THREADS` $\in \{1, 2, 4, 8, 16\}$ (`scripts/bench_threads.sh`); pure compute, no communication.

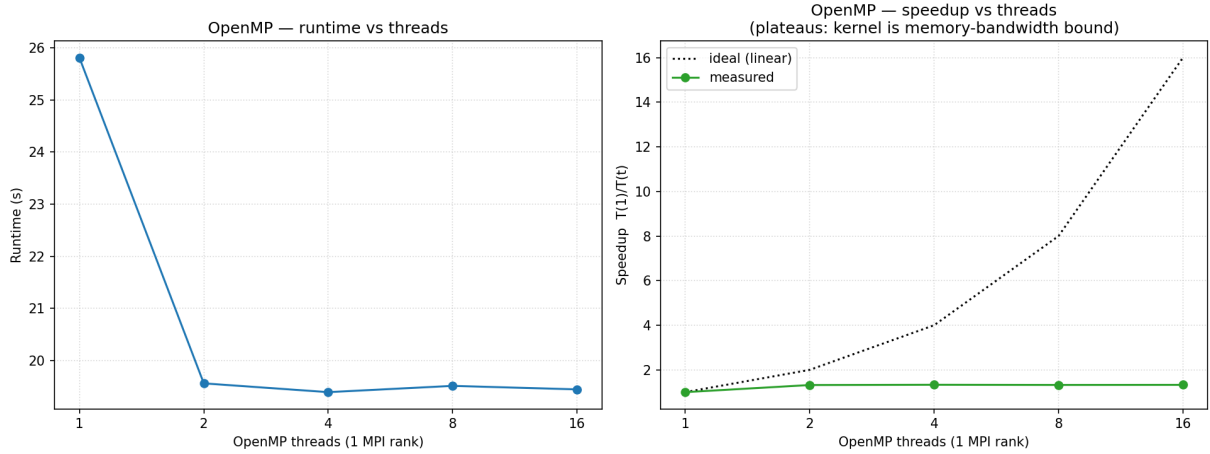


Figure 5: Intra-node OpenMP runtime and speedup vs. thread count ($N = 8192$, **master**).

Observation — a hard memory-bandwidth plateau. The runtime drops from 25.8s at one thread to 19.6s at two ($1.32\times$) and then *does not improve at all* through 4, 8 and 16 threads (19.4–19.5s, speedup stuck at $\approx 1.33\times$). The kernel re-reads the entire K, V block for every query row, so it is *memory-bandwidth bound*: two threads already saturate **master**’s memory channels, and extra threads only contend for the same bandwidth. This is a genuinely useful negative result — it explains why the per-rank (single-thread) MPI decomposition is the right primary lever here, and why the sharded ring path (whose per-rank K, V block is only N/p rows and stays closer to cache) is the more scalable route at large N and large p . It also means a hybrid MPI+OpenMP launch buys little on this kernel; the MPI process dimension, which adds memory channels (and, across nodes, whole separate memory systems), is what actually scales the compute (cf. Chart D, where crossing from one node to three lifts the computation-only speedup from $4.5\times$ on 8 cores to about $20\times$ on 22 cores).

7 Discussion: improvements and remaining limitations

Improvements implemented. Four design choices shaped the final system, each targeting a cost the measurements exposed. The *streaming online-softmax kernel* processes one query row at a time in $\mathcal{O}(N)$ scratch instead of forming the $N \times N$ score matrix, lifting the quadratic-memory

wall so N reaches 65536. The *ring (Flash-style) path* replaces the full- K, V broadcast with $\mathcal{O}(N/p)$ sharded blocks streamed by non-blocking shifts that overlap the compute, cutting cross-node communication $4\times$ and idle wait to near zero (Chart E). The *collective hybrid distribution* fans work to the group roots with **Scatter/Gather** ($\mathcal{O}(\log G)$) rather than a serial **Send/Recv** loop, and the *rank-0-only input allocation* relieves non-root ranks of the unused full matrices, removing the memory ceiling on p . An OpenMP layer over the query-row loop adds an intra-node level beneath MPI, and the built-in profiler separates compute/comm/wait and counts bytes and latency so every run explains its own time.

Limitations. Three remain, and each is visible in the data. The kernel is *memory-bandwidth bound* — it re-reads K, V for every query row, so intra-node threading plateaus at $1.33\times$ (Chart F); cache/query tiling would raise arithmetic intensity but would forgo the current bit-exact match. The compute is *portable scalar C* (auto-vectorized at `-O2`, not tuned BLAS), so absolute throughput is modest — acceptable for a parallelism study. And the measurements carry honest *noise*: they are sampled at rank 0 (which understates the slowest rank’s comm and, at large p , flatters the compute number as rank 0 holds fewer rows), and the Wi-Fi time varies $> 2\times$ between repeats (97s vs 226s for the same $p=22$ broadcast), so we draw conclusions from the qualitative trend rather than any single point. Busy-waiting inside blocking MPI calls also keeps idle ranks at 100% CPU, which the `--profile-wait` timer makes visible (Chart C).

8 Conclusion

We built the Transformer multi-head-attention operator from scratch in C + MPI and parallelized it three ways — head (task), tensor (data; 1D row-block, 2D Cannon, and a ring/Flash-style path), and hybrid (task $\times$ data via `MPI_Comm_split`) — over a single shared streaming kernel, with an OpenMP layer inside each rank. Every path reproduces the sequential baseline within tolerance (bit-exact for 1D/head/hybrid, $\sim 10^{-7}$ for the reordered Cannon/ring paths), so the parallel program is a faithful solution to the problem, and the implementation answers each part of the brief — level of parallelism, decomposition, mapping, communication, load balancing, and pseudocode — with a measured experiment behind every claim.

What the numbers really say. The honest headline is a *gap between two kinds of scaling*. The arithmetic divides almost ideally — computation-only speedup reaches about $20\times$ on 22 cores — but the end-to-end runtime tells a sterner story: over Wi-Fi the best wall-clock speedup is only $4.4\times$, and it is reached at $p = 8$ on a *single* machine. Bringing in the second and third nodes makes the computation faster yet the program slower, because the K, V broadcast and gather must cross a link roughly three orders of magnitude slower than local memory. That same asymmetry turns a statically “balanced” hybrid mapping into a 175.7% load imbalance, which a change of decomposition — to the rootless 1D split — brings down to 28.6%.

A realistic appraisal. Read plainly, *this workload and this cluster are mismatched*, and the report’s worth is in showing precisely why and by how much. Attention is $\mathcal{O}(N^2)$ compute over $\mathcal{O}(N)$ data, so in principle it should be compute-bound and scale well — and it does, on one multi-core node or whenever the data stays local. The moment that data must travel over commodity Wi-Fi the constant factors invert: the operator is communication-bound at every size the 2–3 minute budget permits, and only beyond $N \approx 131072$ (a ~ 30 -minute run) would computation finally dominate. Distributing attention across machines is thus worth it precisely when the interconnect is fast (wired Ethernet or InfiniBand) or the per-node problem is large enough to amortize the transfer; on a shared wireless LAN the single fastest node is hard to beat. The two levers that actually move the result are the ones that *reduce and overlap* data movement (the sharded, non-blocking ring path) and that *place work by capability* (throughput-proportional,

topology-aware distribution on heterogeneous nodes) — not the ones that merely divide the arithmetic more finely.

Closing. The project therefore delivers two things: a correct, framework-free, three-way-parallel attention, and — more valuably — a measured account of where its time goes and which engineering choices change it. The next steps follow directly and change none of the parallel structure, only the three quantities this study isolated. A *wired network* would pull the with-communication curve toward the computation-only one; *FlashAttention-style tiling* would lift the memory-bandwidth ceiling that caps intra-node threading; and *throughput-weighted row assignment* would close the residual heterogeneity imbalance. The communication wall measured here is not a flaw in the code but a property of the machine — and naming it quantitatively is the point of the exercise.

A Reproducing the results

1. **Build:** `make` (produces `hybrid_attention`).
2. **Cluster:** bring up SSH/NFS/OpenMPI and confirm with `bash scripts/test_cluster.sh` (wait for `cluster OK`); see `CHECKLIST.md`.
3. **Correctness:** `mpirun --hostfile hostfile -np X ./hybrid_attention --mode hybrid --seq-len 1024`.
4. **Benchmarks** (set `TOTAL_PROCS` = core count; on our cluster the binary is staged to `/tmp/mpiattn` on each node and the LAN interface is pinned, hence the extra environment variables):

```
export BINARY=/tmp/mpiattn/hybrid_attention NET=192.168.124.0/24 HOSTFILE=
hostfile
TOTAL_PROCS=22 MODE=hybrid TARGET_S=140 bash scripts/bench_n_sizing.sh      #
Chart B
TOTAL_PROCS=22 N=32768 MODE=hybrid      bash scripts/bench_granularity.sh #
Chart C
TOTAL_PROCS=44 N=16384 MODE=tensor PLIST="2 4 8 16 22 32 44" \
bash scripts/bench_speedup.sh                                             #
Chart D
TOTAL_PROCS=16 N=16384 bash scripts/bench_ring_vs_1d.sh                  #
Chart E
MAX_THREADS=16 N=8192  bash scripts/bench_threads.sh                     #
Chart F
```

5. **Charts:** `python3 scripts/make_charts.py` (needs `python3-matplotlib`); writes the five PNGs this report includes.

B Raw per-process measurements (granularity run)

Tables 11 and 12 list every process’s measured times for the two granularity runs of Section 6.4 ($N = 32768$, $p = 22$, `--profile-wait`), the data behind Chart C and the 175.7% $\rightarrow$ 28.6% adjustment. “Work” = compute + comm is the load-balance quantity; “wait” is idle time at barriers. Note in the hybrid table how the two group roots (ranks 0 and 11) carry far more communication than the members, and how the fast **master** ranks accumulate large *wait* while the communication-bound **slave** ranks accumulate little — the signature of a communication-driven imbalance.

Table 11: Hybrid ($G = 2$) per-process times (s). Imbalance on “work”: $(147.2 - 53.4)/53.4 = 175.7\%$.

rank	node	compute	comm	wait	work	rank	node	compute	comm	wait	work
0	master	29.0	48.0	70.7	77.0	0	master	4.8	69.2	5.4	74.0
1	master	35.4	18.0	83.1	53.4	1	master	4.7	63.6	9.7	68.3
2	master	35.8	18.0	82.7	53.8	2	master	4.7	58.8	14.5	63.5
3	master	35.7	18.0	82.8	53.7	3	master	4.8	71.3	1.8	76.2
4	master	35.9	18.0	82.6	53.9	4	master	4.8	68.9	4.3	73.6
5	master	36.2	18.0	82.3	54.2	5	master	4.7	73.2	0.1	77.9
6	master	35.6	18.0	82.8	53.7	6	master	4.7	58.8	14.4	63.5
7	master	35.9	18.0	82.5	54.0	7	master	5.0	71.3	1.6	76.3
8	slave1	30.8	36.8	68.9	67.6	8	slave1	4.4	69.7	3.8	74.1
9	slave1	30.8	39.2	66.6	70.0	9	slave1	5.1	64.3	8.7	69.4
10	slave1	29.9	40.6	66.3	70.5	10	slave1	5.2	63.0	10.0	68.2
11	slave1	13.8	133.4	0.3	147.2	11	slave1	3.5	72.0	2.7	75.5
12	slave2	12.1	73.3	50.1	85.4	12	slave2	1.5	69.5	7.3	71.0
13	slave2	15.7	74.8	45.1	90.5	13	slave2	1.6	74.3	2.6	75.9
14	slave2	8.3	77.6	49.7	85.9	14	slave2	1.7	64.1	12.8	65.7
15	slave2	11.8	79.7	44.3	91.5	15	slave2	1.1	72.7	4.8	73.9
16	slave2	6.5	71.1	58.2	77.7	16	slave2	2.5	68.5	7.8	70.9
17	slave2	6.1	70.2	59.6	76.3	17	slave2	1.8	65.5	11.6	67.3
18	slave2	13.1	53.9	69.1	67.0	18	slave2	1.3	59.3	18.5	60.5
19	slave2	13.9	108.6	13.7	122.5	19	slave2	1.3	72.0	5.9	73.2
20	slave2	15.2	76.5	44.5	91.8	20	slave2	2.2	69.6	7.4	71.8
21	slave2	12.5	58.8	65.0	71.4	21	slave2	1.9	74.6	2.9	77.9

Table 12: 1D tensor per-process times (s). Imbalance on “work”: $(77.9 - 60.5)/60.5 = 28.6\%$.

C Speedup at the doubled size $2N^* = 65536$

The brief specifies the speedup study at $2N^*$. We also ran that size; it is reported here rather than as the headline chart because, as Section 6.5 explains, each point exceeds ten minutes of wall time and the Wi-Fi communication time varies by more than $2\times$ between repeats, so the N^* sweep gives a cleaner, mutually consistent curve. The qualitative behaviour at $2N^*$ is identical (Table 13): the computation-only speedup climbs past $20\times$ while the with-communication speedup is pinned in the low single digits by the network — the same communication wall.

Table 13: Speedup at $2N^* = 65536$ ($T(1) = 401.7$ s, rank-0 timings). Single-sampled over Wi-Fi, so the with-communication column carries visible network noise (e.g. the $p = 22$ dip from a slow broadcast sample); the trend — compute scales, communication caps — matches Chart D.

p	$T(p)$ no comm (s)	speedup (no comm)	$T(p)$ with comm (s)	speedup (with comm)
1	401.65	1.00	401.69	1.00
2	210.35	1.91	210.42	1.91
4	129.34	3.11	130.34	3.08
8	125.45	3.20	126.65	3.17
16	31.61	12.71	107.81	3.73
22	19.07	21.06	245.42	1.64
32	21.42	18.75	66.69	6.02
44	23.20	17.32	74.88	5.36